

DevOps and Database

Oil India Limited - SIH 2026 | PS 26165 - GOARC Safety 1.0 Prototype

Complete Knowledge Base - Database Design to Docker and Deployment

1 - Database - Architecture and Role

Role: Persistent store for every OIL safety report and its AI analysis. Frontend never talks directly to DB - it goes via Backend FastAPI (8000) which uses SQLAlchemy 2 + Pydantic to read/write. Dual-engine: PostgreSQL 16-alpine when running docker compose (production-like) and SQLite (backend/sif.db) when running ./start.sh without Docker Desktop. Both share the same Python models.

Files: database/init.sql (Postgres DDL, runs on first postgres boot via /docker-entrypoint-initdb.d), backend/database.py (engine factory), backend/models.py (8-table ORM), backend/seed.py (50 OIL-style rows from frontend/src/data/reportsData.js).

2 - Schema - 8-Table Normalized Design

Core entity is reports; every AI output is a child of one report. Designed for PS 26165: SIF flag, 9 IOGP LSRs, entities, and clusters.

Table	PK / FK	Key Columns	Notes
reports	id String36 PK report_id UNIQUE R-001	report_date, site (location), activity, report_type, risk, status, reported_by, narrative, batch_id	Business ID R-001, DB PK UUID string
classifications	id PK report_id FK CASCADE	sif_probability, sif_flag, confidence high/medium/low, model_version, explanation_snippets JSON	One per analysis, JSON string for SQLite
lsr_tags	id PK report_id FK	rule_name (9 IOGP + Hydrocarbon Release), confidence, matched_keywords JSON	Many per report, LSR Distribution chart
entities	id PK report_id FK	entity_type activity/location/barrier_failure, entity_value, confidence	NER output
report_clusters	id PK	cluster_label, description, report_count, sif_count	Pattern mining future
report_batches	id PK report_id+cluster_id FK	M:N map, created_at	Cascade delete
uploads	id PK UUID	filename, total_reports, sif_count, processed_at	Tracks CSV uploads
users	id PK email UNIQUE	name, role hse_manager/safety_officer/admin	Auth-ready

ERD: reports 1-N classifications/lsr_tags/entities; reports N-M clusters via report_clusters; reports -> batches via batch_id.

init.sql vs models.py: init.sql uses Postgres UUID/JSONB/uuid-ossf + IF NOT EXISTS; models.py uses String36 + Text JSON string so same code creates tables on SQLite and Postgres.

2.1 Indexes and Query Patterns

Indexes: idx_reports_site, idx_reports_date, idx_reports_batch, idx_classifications_sif_flag, idx_lsr_rule. Used by GET /api/reports (ilike on narrative/report_id/site + sif_only join) and GET /api/dashboard/stats (group_by). Search now uses ilike on all three fields for SQLite/Postgres.

3 - Engine - Dual PostgreSQL / SQLite

backend/database.py reads DATABASE_URL from .env (sqlite:///./sif.db) or docker-compose (postgres://sif_user:sif_pass@postgres:5432/sif_db). SQLite: StaticPool + check_same_thread False (safe for FastAPI threadpool). Postgres: pool_pre_ping True. Lifespan: Base.metadata.create_all on startup; init.sql only on first Docker boot. .env.example documents POSTGRES_DB/USER/PASSWORD, DATABASE_URL, AI_SERVICE_URL, VITE_API_URL.

4 - Seed, Migrations and Operations

backend/seed.py: Parses frontend/src/data/reportsData.js (50 rows) via regex + fallback 5 rows, idempotent (inserts only missing report_id unless --force deletes and re-seeds). Date parsing utils.parse_date 10 formats + dateutil fallback (30 Aug -> 30 August 2026). utils.next_report_id with IntegrityError retry (5x) avoids race. Alembic installed, migrations ready for prod. Backup: postgres_data volume + pg_dump; SQLite copy sif.db; Monitoring: GET /health, /health/db SELECT 1, /health/ai.

5 - DevOps - Philosophy and Modes

Goal: One codebase runs without Docker Desktop via ./start.sh (SQLite) and with Docker via docker compose up (Postgres). Both use 5173/8000/8001/5432 and GOARC theme + Ferrari OIL splash (56x56, 700ms in, 1.4s hold, 650ms fade, sessionStorage).

Mode	Command	DB	AI	When
Local	bash ./start.sh	SQLite backend/sif.db	uvicorn 8001 Python 3.12 lazy	Hackathon demo
Docker	docker compose up --build	postgres:16-alpine 5432	uvicorn 8001 python:3.11-slim + libgomp1	OIL on-prem

6 - Dockerfiles - Per Service

Backend Dockerfile (python:3.11-slim): apt gcc libpq-dev, pip install, COPY, EXPOSE 8000, CMD uvicorn 8000

AI Dockerfile (python:3.11-slim plus libgomp1): pip install, EXPOSE 8001, CMD uvicorn 8001

Frontend Dockerfile (node:20-alpine): npm ci, EXPOSE 5173, CMD npm run dev host 0.0.0.0

frontend/vite.config.js: server port 5173 host 0.0.0.0 required for Docker. **.dockerignore** hides .env, node_modules, __pycache__, *.db.

7 - docker-compose.yml - Orchestration

Services: postgres (postgres:16-alpine, sif-postgres, env POSTGRES_*, volume postgres_data + init.sql, healthcheck pg_isready), backend (build ./backend, env DATABASE_URL postgres, AI_SERVICE_URL http://ai:8001, volumes ./backend:/app + /app/__pycache__ + /app/.env anonymous to avoid host shadowing, depends_on postgres service_healthy, command uvicorn --reload), ai (volumes ./ai:/app + ./data:/data), frontend (env VITE_API_URL http://localhost:8000, 5173:5173, /app/node_modules anon, npm run dev --host). Version key removed.

8 - start.sh - No-Docker Local Dev

Steps: (1) backend venv Python 3.12 pip install aiosqlite, (2) ai venv pip install binary wheels, (3) frontend npm ci, (4) nohup uvicorn 8000+8001 + npm dev, (5) wait-for-health curl /health 15x, (6) python seed.py idempotent, (7) trap kill on Ctrl+C. Ports 5173/8000/8001/5432. Fixed: Python 3.14 -> 3.12, relaxed ai pins to avoid blis, 5173 vs 3000, PAT redaction.

9 - Environment, Secrets and Health

.env.example documents POSTGRES_DB/USER/PASSWORD, DATABASE_URL, AI_SERVICE_URL, VITE_API_URL. **.env** local uses sqlite and is gitignored; PAT injected via https://token@github for push then reset. **Health:** GET /health, /health/db SELECT 1, /health/ai (httpx to AI). README Quick Start shows both modes.

10 - CI/CD and Deployment Roadmap

Current: Manual git push main (rebase), no Actions yet. **Next:** GitHub Actions lint, pytest, docker build, push GHCR, DevOps merges develop->main, cloud free tier with postgres_data volume, pg_dump backup, Uvicorn workers via gunicorn + Redis for stats + Celery for batch, Prometheus + Grafana, HTTPS + JWT + RBAC.

11 - Security, Pitfalls Fixed and Gotchas

Fixed: Flask 8000 conflict, frontend 3000->5173, host venv shadowing, ilike, report_id race with retry, AI path dual resolve + lazy load, blis build with binary wheels, CORS, 405 route. **Remaining:** Alembic migrations pending (create_all suffices), single uvicorn process, batch sync timeout -> queue next.

Stack: React 19 + Vite + SQLAlchemy + sklearn 1.9 + PostgreSQL/SQLite + Docker Compose - Ports 5173/8000/8001/5432 - GOARC #FCAB04 - OIL Safety 1.0
Prototype
yugantyadav/OIL-SIF-PRECURSOR - 30 Aug 2026